

Diagramas de sintáxis

Necesitamos que la gramática esté en notación **EBNF**, por lo que, nos queda de la siguiente manera:

```
programa → decl_proto decl_var decl_func

decl_proto → [ proto tipo id ( argumentos ) ; { proto tipo id ( argumentos ) ; } ]

decl_var → [ tipo lista_var ; { tipo lista_var ; } ]

tipo → basico compuesto
      | struct { decl_var }
      | puntero

puntero → ptr basico

basico → int | float | double | complex | rune | void | string

compuesto → [ "[" literal_entera "]" { "[" literal_entera "]" } ]

lista_var → id { "," id }

decl_func → [ func tipo id ( argumentos ) bloque { func tipo id ( argumentos ) bloque } ]

argumentos → [ lista_args ]

lista_args → tipo id { "," tipo id }

bloque → { declaraciones instrucciones }

instrucciones → sentencia { sentencia }

sentencia → parte_izquierda = exp ;
           | if ( exp ) sentencia
           | if ( exp ) sentencia else sentencia
           | while ( exp ) sentencia
           | do sentencia while ( exp )
           | break ;
           | bloque
           | return exp ;
           | return ;
           | switch ( exp ) { casos }
           | print exp ;
           | scan parte_izquierda

casos → { caso } [ predeterminado ]

caso → case opcion : instrucciones

opcion → literal_entera | literal_runa

predeterminado → default : instrucciones

parte_izquierda → id [ localizacion ]

exp → exp "||" exp
     | exp "&&" exp
```

```

| exp "==" exp
| exp "!=" exp
| exp "<" exp
| exp "<=" exp
| exp ">=" exp
| exp ">" exp
| exp "+" exp
| exp "-" exp
| exp "*" exp
| exp "/" exp
| exp "%" exp
| exp "//" exp
| "!" exp
| "-" exp
| "(" exp ")"
| id [ localizacion ]
| false
| literal_cadena
| true
| literal_runa
| literal_entera
| literal_flotante
| literal_doble
| literal_compleja
| id ( parametros )
| id

```

```
parametros → [ lista_param ]
```

```
lista_param → exp { "," exp }
```

```
localizacion → arreglo
               | estructurado
```

```
arreglo → "[" exp "]" { "[" exp "]" }
```

```
estructurado → "." id { "." id }
```

De acuerdo a la notación aceptada por Railroad Diagram Generator, ahora nuestra gramática se verá de la siguiente forma:

```
programa ::= decl_proto decl_var decl_func
```

```
decl_proto ::= ("proto" tipo "id" "(" argumentos ")" ";" )*
```

```
decl_var ::= (tipo lista_var ";" )*
```

```
tipo ::= basico compuesto
        | "struct" "{" decl_var "}"
        | puntero
```

```
puntero ::= "ptr" basico
```

```
basico ::= "int" | "float" | "double" | "complex" | "rune" | "void" | "string"
```

```
compuesto ::= "[" literal_entera "]" )*
```

```

lista_var ::= "id" ("," "id")*

decl_func ::= ("func" tipo "id" "(" argumentos ")" bloque )*

argumentos ::= lista_args?

lista_args ::= tipo "id" ("," tipo "id")*

bloque ::= "{" declaraciones instrucciones "}"

instrucciones ::= (sentencia )*

sentencia ::= parte_izquierda "=" exp ";"
            | "if" "(" exp ")" sentencia
            | "if" "(" exp ")" sentencia "else" sentencia
            | "while" "(" exp ")" sentencia
            | "do" sentencia "while" "(" exp ")" ";"
            | "break" ";"
            | bloque
            | "return" exp ";"
            | "return" ";"
            | "switch" "(" exp ")" "{" casos "}"
            | "print" exp ";"
            | "scan" parte_izquierda

casos ::= caso* predeterminado?

caso ::= "case" opcion ":" instrucciones

opcion ::= literal_entera | literal_runa

predeterminado ::= "default" ":" instrucciones

parte_izquierda ::= "id" localizacion?

exp ::= exp "||" exp
      | exp "&&" exp
      | exp "==" exp
      | exp "!=" exp
      | exp "<" exp
      | exp "<=" exp
      | exp ">" exp
      | exp ">=" exp
      | exp "+" exp
      | exp "-" exp
      | exp "*" exp
      | exp "/" exp
      | exp "%" exp
      | exp "//" exp
      | "!" exp
      | "-" exp
      | "(" exp ")"
      | "id" localizacion?
      | "false"
      | "true"

```

```

| literal_cadena
| literal_runa
| literal_entera
| literal_flotante
| literal_doble
| literal_compleja
| "id" "(" parametros ")"
| "id"

parametros ::= lista_param?

lista_param ::= exp ("," exp)*

localizacion ::= arreglo
               | estructurado

arreglo ::= "[" exp "]" ("[" exp "]" )*

estructurado ::= "." "id" ( "." "id" )*

```

Una vez ingresada, obtenemos los diagramas: